

```
STATUS

ACCOUNT

STATUS

class Banner
  attr_accessible :horiz, :link, :visible, :image, :position
  has_attached_file :image, styles: { vert: '220'

  before_create :assign_position

  def assign_position
    max = Banner.maximum(:position)
    self.position = max ? max + 1 : 0
  end
end
```

FULL STACK DEVELOPMENT
REACT – O PRINCÍPIO

AULA 07

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON.....	4
SAIBA MAIS	5
O QUE VOCÊ VIU NESTA AULA?	12
REFERÊNCIAS	13

EMANSP

O QUE VEM POR AÍ?

Nesta aula, focaremos em expandir nosso conhecimento sobre as funcionalidades avançadas do React Router. Abordaremos como utilizar hooks personalizados do React Router para uma gestão mais precisa do estado da navegação. Também exploraremos técnicas para reforçar a segurança nas rotas, assegurando que apenas usuários devidamente autorizados tenham acesso a partes específicas da aplicação. Esses conceitos são essenciais para aprimorar a construção de aplicações SPA, equipando-nos para lidar com questões de escalabilidade e segurança de forma efetiva.

HANDS ON

Neste vídeo, demonstraremos como utilizar o React Router para criar uma aplicação Single Page Application (SPA) eficiente e interativa. Através de um projeto prático, veremos como instalar e configurar o React Router, definindo rotas para diferentes páginas como Home, Sobre e Contato, e também como manipular parâmetros de URL para exibir detalhes específicos, como os de uma palestra. Além disso, será abordada a navegação programática e o uso do histórico do navegador para permitir que os usuários naveguem com os botões de voltar e avançar sem recarregar a página, mantendo uma experiência de usuário fluida e responsiva. Acompanhe o passo a passo para aprender a implementar essas funcionalidades em seu próprio projeto React, explorando o potencial do React Router para desenvolver SPAs modernas e eficazes.

SAIBA MAIS

O React Router é uma biblioteca fundamental no desenvolvimento de aplicações web modernas, especialmente quando falamos de Single Page Applications (SPAs). Single Page Applications representam um paradigma no desenvolvimento web, visando melhorar a experiência do usuário em sites e aplicações. Em vez de carregar novas páginas do servidor cada vez que o usuário solicita uma nova seção ou página de um site, uma SPA carrega uma única página HTML no início e, em seguida, atualiza o conteúdo dessa página dinamicamente conforme o usuário interage com a aplicação. Esse modelo de carregamento e atualização é facilitado por JavaScript e frameworks ou bibliotecas específicas, como o React, Angular ou Vue.js.

Funcionamento básico

O funcionamento de uma SPA se baseia na manipulação dinâmica do DOM (Document Object Model) através de JavaScript. Quando o usuário interage com elementos da página, como links e botões, em vez de solicitar uma nova página ao servidor, a aplicação web requisita apenas os dados necessários, geralmente através de APIs em formato JSON, e atualiza a página atual com esses novos dados. Isso elimina a necessidade de recarregar toda a página, resultando em uma experiência mais rápida e suave para o usuário.

Vantagens

- Desempenho melhorado: uma vez que recursos como CSS e JavaScript são carregados apenas uma vez, no início, as interações subsequentes são mais rápidas, pois apenas os dados necessários são transmitidos entre o cliente e o servidor.
- Experiência de usuário aprimorada: a transição entre conteúdos é mais fluida, sem os habituais tempos de espera para o carregamento de novas páginas, criando uma experiência semelhante à de um aplicativo nativo.

- Desenvolvimento simplificado: do ponto de vista da pessoa desenvolvedora, SPA permite uma separação clara entre o frontend e o backend, facilitando o desenvolvimento e a manutenção.

Desafios

- SEO (Search Engine Optimization): as SPAs podem apresentar desafios para a otimização de mecanismos de busca, pois os crawlers de busca tradicionais podem ter dificuldade em indexar conteúdo dinâmico. No entanto, estratégias como Server-Side Rendering (SSR) ou prerendering podem ajudar a mitigar esse problema.
- Gerenciamento de estado: manter o estado da aplicação pode se tornar complexo, especialmente à medida que a aplicação cresce. Bibliotecas de gerenciamento de estado, como Redux (para React), podem ajudar a organizar e gerenciar o estado de forma mais eficiente.
- Navegação e histórico: gerenciar o histórico de navegação e permitir que o usuário use os botões de avançar e voltar do navegador de forma intuitiva requer atenção especial em SPAs. Isso é onde o React Router e bibliotecas similares se tornam indispensáveis, ao facilitar o gerenciamento de rotas na aplicação.

Em resumo, as SPAs transformaram a maneira como desenvolvemos e interagimos com aplicações web, oferecendo uma experiência mais rica e responsiva. Com as ferramentas e estratégias certas, profissionais de desenvolvimento podem superar os desafios associados a SPAs, aproveitando seus benefícios para criar aplicações modernas e eficientes.

O React Router desempenha um papel crucial nesse cenário ao gerenciar as rotas (ou seja, os diferentes caminhos ou URLs) dentro de uma aplicação React. Ele permite que os(as) desenvolvedores(as) definam diferentes "páginas" ou componentes que devem ser exibidos com base na URL, sem a necessidade de recarregar a página. Isso é feito manipulando o histórico do navegador de forma programática, para que os usuários possam usar botões de avançar e voltar do navegador para navegar entre essas "páginas" simuladas.

Exemplo prático

Vamos começar com um exemplo básico de como usar o React Router em uma aplicação React. Primeiramente, é necessário instalar o React Router no seu projeto, utilizando o npm ou yarn:

```
npm install react-router-dom
```

ou

```
yarn add react-router-dom
```

Após a instalação, você pode começar a usar o React Router definindo rotas e componentes correspondentes. Aqui está um exemplo simples de uma aplicação com duas páginas, Home e About:

```
import React from "react";
import { BrowserRouter as Router, Route, Switch, Link } from "react-router-dom";

const Home = () => <div>Home Page</div>;
const About = () => <div>About Page</div>;

function App() {
  return (
    <Router>
      <div>
        <nav>
          <ul>
            <li>
              <Link to="/">Home</Link>
            </li>
            <li>
              <Link to="/about">About</Link>
            </li>
          </ul>
        </nav>

        <Switch>
          <Route path="/" component={Home} />
          <Route path="/about" component={About} />
        </Switch>
      </div>
    </Router>
  );
}
```

```

    <Route path="/about">
      <About />
    </Route>
    <Route path="/">
      <Home />
    </Route>
  </Switch>
</div>
</Router>
);
}

```

Neste exemplo, utilizamos BrowserRouter como o componente de mais alto nível que envolve nossa aplicação React. Dentro dele, definimos um componente nav simples para navegação, utilizando o componente Link para criar links navegáveis que não recarregam a página. O componente Switch é usado para renderizar exclusivamente o primeiro Route que corresponde à URL atual. Cada Route especifica um caminho (path) e um componente (component) que deve ser renderizado quando o caminho corresponde à URL.

Avançando com parâmetros de URL

React Router também permite definir parâmetros dinâmicos nas rotas, o que é útil para criar páginas detalhadas baseadas em IDs ou outros identificadores. Aqui está um exemplo de como usar parâmetros de URL:

```

import React from "react";
import { BrowserRouter as Router, Route, Switch, Link, useParams } from "react-router-dom";

const ItemDetail = () => {
  let { id } = useParams();
  return <div>Item ID: {id}</div>;
};

function App() {
  return (
    <Router>
      <div>
        <nav>
          <ul>

```



```

    <li>
      <Link to="/item/1">Item 1</Link>
    </li>
    <li>
      <Link to="/item/2">Item 2</Link>
    </li>
  </ul>
</nav>

<Switch>
  <Route path="/item/:id">
    <ItemDetail />
  </Route>
</Switch>
</div>
</Router>
);
}

```

Neste caso, `:id` é um parâmetro de URL que o React Router automaticamente extrai e disponibiliza via o hook `useParams`. Isso permite que o componente `ItemDetail` acesse o id correspondente e utilize-o para buscar ou exibir informações detalhadas sobre um item específico.

O React Router oferece ainda mais funcionalidades, incluindo rotas aninhadas, redirecionamentos e a customização do comportamento de roteamento para atender necessidades específicas de complexas aplicações web. Por meio da combinação dessas técnicas, as pessoas desenvolvedoras podem criar experiências de usuário ricas e interativas que são rápidas, responsivas e intuitivas.

Rotas aninhadas

Rotas aninhadas permitem que você crie uma hierarquia de componentes baseada na estrutura de URLs de sua aplicação. Isso é especialmente útil em aplicações grandes, onde diferentes seções da aplicação possuem seus próprios subcomponentes e sub-rotas. Com o React Router, você pode definir rotas dentro de rotas, o que reflete a estrutura aninhada da sua UI e da URL.

Por exemplo, considere uma aplicação que possui uma página de usuário e, dentro dela, uma subpágina para editar o perfil do usuário. A estrutura de rotas aninhadas pode ser implementada da seguinte forma:

```
<Router>  
  <Route path="/user" component={User}>  
    <Route path="edit" component={EditProfile} />  
  </Route>  
</Router>
```

Neste exemplo, ao navegar para `/user/edit`, o componente `User` seria renderizado juntamente com o componente `EditProfile`. Essa abordagem mantém a organização do código alinhada com a estrutura da URL e da UI, facilitando a manutenção e a compreensão do aplicativo.

Redirecionamentos

Redirecionamentos são essenciais para guiar os usuários através do fluxo da aplicação ou redirecioná-los para páginas específicas em determinadas condições (como redirecionar um usuário não autenticado para a página de login). O React Router oferece o componente `Redirect` para facilitar essa tarefa. Por exemplo, para redirecionar um usuário automaticamente da raiz do site para a página inicial, você poderia usar:

```
<Switch>  
  <Route path="/home" component={HomePage} />  
  <Redirect from="/" to="/home" />  
</Switch>
```

Comportamento de roteamento

O comportamento de roteamento pode ser customizado para atender necessidades específicas, como o carregamento preguiçoso de componentes, que é uma técnica para carregar componentes sob demanda, melhorando o tempo de carregamento inicial da aplicação. O React Router suporta essa funcionalidade por meio da função `React.lazy` e do componente `Suspense` do React.

Um exemplo de carregamento preguiçoso seria:

```
import React, { Suspense, lazy } from 'react';
import { BrowserRouter as Router, Route, Switch } from 'react-router-dom';

const HomePage = lazy(() => import('./HomePage'));
const AboutPage = lazy(() => import('./AboutPage'));

function App() {
  return (
    <Router>
      <Suspense fallback=<div>Carregando...</div>>
        <Switch>
          <Route path="/about" component={AboutPage} />
          <Route path="/" component={HomePage} />
        </Switch>
      </Suspense>
    </Router>
  );
}
```

Neste caso, HomePage e AboutPage são carregados apenas quando necessários, o que pode representar uma melhora significativa no desempenho para aplicações grandes.

Adotar essas técnicas avançadas permite aos(as) desenvolvedores(as) criar aplicações web ricas e interativas com o React Router, melhorando não apenas a experiência do usuário, mas também a eficiência do desenvolvimento. A capacidade de criar rotas aninhadas, implementar redirecionamentos de forma intuitiva e customizar o comportamento de roteamento para otimizar o desempenho torna o React Router uma ferramenta indispensável no ecossistema React.

O QUE VOCÊ VIU NESTA AULA?

Nesta aula, exploramos o conceito de Single Page Applications (SPAs) e o papel vital do React Router no desenvolvimento dessas aplicações web modernas. Aprendemos que SPAs carregam uma única página HTML e atualizam seu conteúdo dinamicamente, sem a necessidade de recarregar a página, criando uma experiência de usuário fluida e “aplicativo-like”. Discutimos como o React Router facilita o roteamento dentro de aplicações React, permitindo a definição de diferentes “páginas” ou componentes com base na URL, sem recarregar a página. Examinamos exemplos práticos que incluíram a instalação do React Router, a criação de rotas simples, a implementação de rotas aninhadas, redirecionamentos e o carregamento preguiçoso de componentes para otimizar o desempenho. Por meio dessas técnicas, destacamos como o React Router permite que pessoas desenvolvedoras construam sistemas de navegação eficientes, melhorando a organização do código, a manutenção, a escalabilidade e, finalmente, a experiência do usuário em aplicações web complexas.

REFERÊNCIAS

ALURA. **React Native: fazendo requisições com Fetch**. 2021. Disponível em: <https://www.alura.com.br/artigos/react-native-fazendo-requisicoes-fetch>. Acesso em: 01 jul. 2024.

ALURA. **React: desenvolvendo em React Router com JavaScript**. 2022. Disponível em: <https://www.alura.com.br/conteudo/React-desenvolvendo-react-router-javascript>. Acesso em: 01 jul. 2024.

REACT. **Documentação React**. [s.d.]. Disponível em: <https://react.dev/>. Acesso em: 01 jul. 2024.

PALAVRAS-CHAVE

Palavras-chave: ReactJs. Front-end. Biblioteca. React-hooks.

EMSE

POS TECH